

Project documentation: Extractive summation of Bulgarian texts (TextRank and LSA)

Course: Natural Language Processing Approaches

1. Task Selection (Introduction)

The aim of the project is to automatically summarize Bulgarian news texts in an extractive manner. Extractive summarization selects the most representative sentences from the original text and arranges them into a short summary. Two independent approaches – a graph method (TextRank) and a thematic method (Latent Semantic Analysis (LSA)) – were implemented to compare their behavior on the same corpus and pre-processing.

2. Enclosure selection and additional data (Data)

The input data is news texts in Bulgarian in CoNLL format, where each token is annotated with a Part of Speech (POS) tag and a Named Entity Recognition (NER) tag. The project uses sample files from the *data/raw/CoNLL/* directory.

Additional resources:

- a list of stop words from the NLP course (*resources/BTB-StopWordList.txt*);
- Bulgarian Stem Rules (*src/bulstem/stemrules/*) used by BulStem.
- list of reference (gold) summaries generated with ChatGPT (*evaluation/chatgpt/*)

3. Choosing an approach (Method)

3.1 Basic Processing in Computational Linguistics

The pre-processing is performed by reading the CoNLL file (*src/CoNLLReader.py*):

- 1) Separation of sentences.
- 2) Normalization: lowercasing.
- 3) Removing stop words.
- 4) Stemming via BulStem (reduces the influence of the rich Bulgarian morphology).
- 5) Filtering by POS (strategy set): by default, *{NOUN, PROP, ADJ, VERB}* are used to keep the content words. The two other default strategy sets are *{NOUN, PROP, ADJ}* and *{NOUN, PROP}*.

The result for each sentence is a bag-of-words dictionary from stems to frequencies (*edited_dicts*), as well as statistics on POS and NER, which are later used for priorities (*sentence_priors*).

3.2 Sentence priors (bias towards informative sentences)

In order to encourage factually dense sentences, a prior-value is calculated for each sentence based on:

- POS profile (e.g. more nouns/proper names -> higher informativeness);
- NER profile (presence of named units – persons/locations/organizations, etc.).

The combined value is normalized to obtain a probabilistic distribution of *priors* over sentences and is calculated using the formula: $\text{prior} = (\text{ALPHA} * \text{pos_score}) + ((1 - \text{ALPHA}) * \text{ner_score})$

These *priors* are used in both methods, but in different ways.

3.3 Biased TextRank (graph approach)

TextRank models sentences as nodes in a graph. The weight of an edge between two sentences is determined by the cosine proximity between their bag-of-words vectors (based on stem frequencies). The resulting similarity matrix is normalized in rows to a transition matrix P (Markov chain). A custom PageRank (power iteration) is then applied: $w_{\text{new}} = (1-d) * \text{priors} + d * (P^T * w)$ where d is the damping factor (e.g. 0.85). Thus, the algorithm combines:

- "centrality" in the column (sentences similar to many others);
- addition to informative sentences through *priors* (teleport distribution).

3.4 Weighted LSA (thematic approach)

The LSA constructs a term-sentence matrix A (TF-IDF) from edited_dicts. Then the SVD decomposition $A \approx U * S * V^T$ is applied. In our implementation, *priors* are injected as weights by columns (sentences) before the SVD, which reinforces the influence of informative sentences on latent topics.

4. Data experiments (Experiments)

✦ The main experimental setting is as follows:

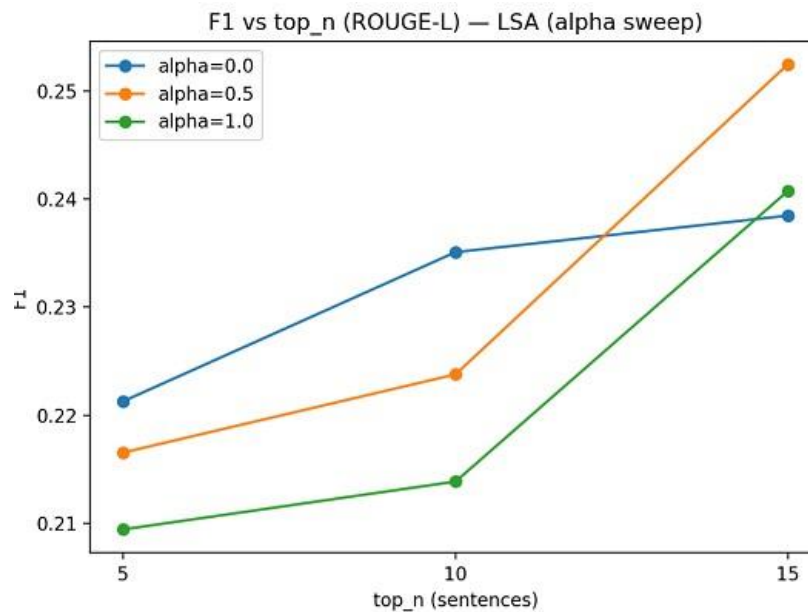
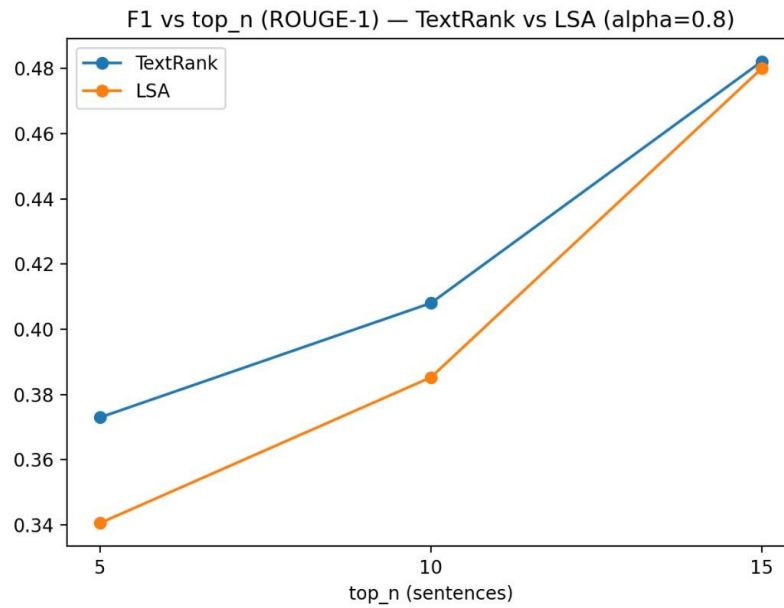
- 1) Select an input CoNLL file.
- 2) Both approaches are executed on the same pre-processed data.
- 3) Top-N sentences are chosen and then arranged in their original order for readability.
- 4) The overlap between the selected sentences from the two methods is calculated.

† Single evaluation statement

1) Score with ROUGE-1/2/L vs. ChatGPT gold summaries and Aggregation of Precision, Recall, and F1 for TextRank and LSA for a single text

† Overall Experimental/Estimating Setup:

1) Score with ROUGE-1/2/L against ChatGPT gold summaries and aggregate the F1 scores for N and $\text{ALPHA} \in \{0, 0.5, 1\}$ for all texts from the database, generating graphs.



4.1 Project structure and main modules

The project is organized into the following directories and files:

- src/: basic logic (CoNLLReader.py – read/processing; TextRank.py – graph ranking; LSA.py – SVD/topic ranks; main.py – compare_summaries and writing summaries).
- run_experiment.py: batch execution on all CoNLL files, ROUGE evaluation, F1 results collection, and graph generation.
- data/: input data (data/raw/CoNLL/*.conll).
- resources/: additional resources (stop words).
- evaluation/chatgpt/: Reference (gold) summaries from ChatGPT with names <filename>_{5|10|15}.txt.
- evaluation/system/: generated system summaries by text, ALPHA and N (TextRank and LSA).
- evaluation/rouge_eval.py: implementation of ROUGE-1, ROUGE-2 and ROUGE-L.
- plots/rouge_curves/: 9 graphs (by metric) to compare F1 curves.

Start:

- Installing dependencies: pip install -r requirements.txt
- One-time summary and overlap: python -m src.main (or python src/main.py from the root directory) and call compare_summaries(...).
- Batch experiment (10 texts, $N \in \{5, 10, 15\}$, $ALPHA \in \{0, 0.5, 1\}$): python run_experiment.py
- Optional: change the global TOP_N to run_experiment.py (it should be 5, 10 or 15).

5. Validation of results and analysis of errors (Results, Discussion)

Quality is automatically evaluated with ROUGE metrics (evaluation/rouge_eval.py), which measure the lexical overlap between a system summary (candidate) and a reference summary (gold). The reference summaries are located in evaluation/chatgpt/ and are named as <conll_basename>_{5|10|15}.txt, so that when batch run, run_experiment.py automatically selects the correct gold file according to the current N. For each text, ROUGE-1, ROUGE-2 and ROUGE-L (Precision/Recall/F1) are calculated, storing the F1 values for TextRank and LSA at different N and ALPHA. Finally, 9 graphs (3 types) with curves of F1 (by metric) are generated: (1) TextRank vs LSA at default ALPHA, (2) ALPHA sweep for TextRank, (3) ALPHA sweep for LSA.

Used:

- ROUGE-1: 1-gram overlap (words) – whether the keywords are present.
- ROUGE-2: 2-gram overlap – phrase/local context sensitive.
- ROUGE-L: based on the longest common subsequence (LCS) – reflects sequence similarity.

For each metric, Precision, Recall, and F1 are calculated.

Interpretation: in extractive summaries, it is normal for precision to be lower if gold is abstract (retelling), because the system includes whole sentences and more

context. For a more honest comparison, it is recommended that gold is the same length (number of sentences) as the system summary.

Typical sources of errors:

- Stemming and stop words: noisy tokens (e.g. abbreviations) are possible to affect similarity.
- NER annotation: errors in BIO tags affect *priors* directly.
- Sentence order: TextRank optimizes centrality, LSA optimizes topic representativeness; so they can choose similar sentences, but in a different rank.

6. Conclusion and future work

The project demonstrates two classic approaches to extractive summarization and adaptation to Bulgarian through stemming, stop-words and the use of POS/NER as *priors*. Future improvements:

- 1) Reduction of repetition: post-processing with Maximal Marginal Relevance (MMR) or punishment of similar sentences.
- 2) Richer representations: TF-IDF with synonymous extensions or embedding-based proximity.
- 3) More objective assessment: more than one gold referent (several human abstracts) and stemmed ROUGE for Bulgarian.
- 4) Hybrid ensemble: combining the ranks from TextRank and LSA (e.g. arithmetic mean or voting).